

10 difficulty levels according to the number of facts (k) required to derive the final result. Note that the knowledge base for kinship compositionality, such as "father's father is grandfather" is not included in the dataset. We, therefore, designed three experiments to help alleviate the issue of missing knowledge base, a) learning with manually specified rules, b) rule learning, and c) learning without rules. We showcase our Scallop code in Fig. 30.

Learn with Manually Specified Rules. A straightforward strategy to overcome the missing required database issue, is to manually add it. We created an external knowledge base with 92 kinship composition triplets and 3 rules. To define how two known relations can be connected to derive the third relation, we design a high-order predicate, *composition*. As an example, the rule "father's mother is grandmother" is encoded as *composition*(FATHER, MOTHER, GRANDMOTHER). The full Scallop program connecting the knowledge base into the reasoning process is shown in Fig. 30. Given the knowledge base, context, and question-answer pairs, we are ready to perform learning over the new dataset. As the context has varies lengths, we first separate the context into multiple windows to ensure the perception model processes similar-length-context across the dataset. Then a large language model, such as RoBERTa [Liu et al. 2019], is adopted to extract relations from these windowed texts. The kinship extracted from the context goes into *kinship*(rela, sub, obj). For example, *kinship*(SON, "Alice", "Bob") indicates that Bob is Alice's son. The query, on the other hand, is stored in the relation *question*(sub, obj). Next, we will combine the predicted relations from all the context windows into one probabilistic database, together with the knowledge base, and the query to perform logical reasoning. Last, we compare the probabilistic query result with the ground truth with BCE loss.

Rule Learning. Specifying 92 composition rules manually could be time consuming. With Scallop, it is possible to do this in a smarter way. Since there are 20 basic kinship relations, we have a finite space containing $20^3 = 8K$ possible composition facts. We treat all 8K composition facts as probabilistic and to be learnt, and therefore we store them inside a tensor of size $20 \times 20 \times 20$. We can initialize this tensor randomly but with a low maximum weight (i.e. 0.1). During training, we will allow gradients to be back-propagated to this composition tensor so that these weights can also be learnt. In this case, the user does not specify any composition fact manually and everything is learnt on the fly. Note that, 8K composition facts can slow down the training time. Therefore, we use multinomial sampling to pick 150 composition facts for training. During testing, we simply select the top 150 for inference.

Learn without Rules. We can also jointly extract entity relations and learn the composition rules without the human specified rules. This is a more challenging task where one needs to figure out the way to combine inferred relations and derive the desired answer without intermediate supervision.

Experimental Setup. We have 10K training data points, which are triplets of passage, query, and answers. We use RoBERTa as the backbone language model. We further train a 2-layer MLP-based relation extractor which takes in the embedding of three components, a global embedding of the passage, and the embedding of the queried subject and the object, and returns the result for a 21-way classification. We set the batch size to 32, the learning rate to 0.00001, and train the models for 100 epochs.

C.6 Mugen

Experimental Setup. We sample 1K Mugen [Hayes et al. 2022] video and text pairs as the training dataset, and another 1K for testing. Our neural module consists of the S3D image embedding and

```

1  // Input from neural networks
2  type action(usize, String)
3  type expr(usize, String)
4  type expr_start(usize)
5  type expr_end(usize)
6  type action_start(usize)
7  type action_end(usize)
8
9  type match_single(usize, usize, usize)
10 type match_sub(usize, usize, usize, usize)
11
12 // Check whether does a subsection of text specification matches the
13 // video content
14 rel match_single(tid, vid, vid + 1) = expr(tid, a), action(vid, a)
15 rel match_sub(tid, tid, vid_start, vid_end) =
16     match_single(tid, vid_start, vid_end)
17 rel match_sub(tid, tid, vid_start, vid_end) =
18     match_sub(tid, tid, vid_start, vid_mid),
19     match_single(tid, vid_mid, vid_end)
20 rel match_sub(tid_start, tid_end, vid_start, vid_end) =
21     match_sub(tid_start, tid_end - 1, vid_start, vid_mid),
22     match_single(tid_end, vid_mid, vid_end)
23
24 // Check whether does the whole text specification matches the video content
25 rel match() = expr_start(tid_start),
26     expr_end(tid_end), action_start(vid_start), action_end(vid_end),
27     match_sub(tid_start, tid_end, vid_start, vid_end)
28
29 // Integrity violation when too many identical text expressions
30 // occurs consecutively
31 rel too_many_consecutive_expr() = expr(tid, a),
32     expr(tid + 1, a), expr(tid + 2, a), expr(tid + 3, a)

```

Fig. 31. Scallop code for Mugen.

the distillBert text embedding. Then we pass the embeddings through a 2-layer MLP, with a hidden layer size of 256. We trained the Scallop module 1000 epochs on the training dataset, the learning rate is set to 0.0001, and the batch size is 3, with BCE-loss closing the training loop. The Scallop code is shown in Fig. 31

C.7 CLEVR

CLEVR [Johnson et al. 2017] stands for **C**ompositional **L**anguage and **E**lementary **V**isual **R**easoning, is a synthetic dataset testing model’s ability to perform *Visual Question Answering* (VQA). Each data point of the dataset contains an image and a question-answer pair with regard to the image. The images are randomly generated from a dictionary of 3 shapes, 8 colors, 2 materials, and 2 sizes and the programmatic queries are also randomly generated. Given a rendered image with simple objects such as cubes and spheres, we aim to answer a question about the image.

Architecture. The object feature vectors are obtained by a 4-layer convolutional neural network. The feature vectors are then passed to four 2-layer MLP classifiers and another 3-layer MLP classifier

```

1  // yes/no question
2  rel eval_yn(e, x > y) = greater_than_expr(e, a, b),
3                          eval_num(a, x), eval_num(b, y)
4  rel eval_yn(e, x < y) = less_than_expr(e, a, b),
5                          eval_num(a, x), eval_num(b, y)
6  rel eval_yn(e, x == y) = equal_expr(e, a, b),
7                          eval_num(a, x), eval_num(b, y)
8  rel eval_yn(e, x == y) = equal_color_expr(e, a, b),
9                          eval_query(a, x), eval_query(b, y)
10 rel eval_yn(e, x == y) = equal_material_expr(e, a, b),
11                          eval_query(a, x), eval_query(b, y)
12 rel eval_yn(e, x == y) = equal_shape_expr(e, a, b),
13                          eval_query(a, x), eval_query(b, y)
14 rel eval_yn(e, x == y) = equal_size_expr(e, a, b),
15                          eval_query(a, x), eval_query(b, y)
16 rel eval_yn(e, b) = b := exists(o: eval_objs(f, o)
17                      where e: exists_expr(e, f))
18
19 // count number of objects
20 rel eval_num(e, n) = n := count(o: eval_objs(f, o) where e: count_expr(e, f))
21
22 // objects filter
23 rel eval_objs(e, o) = scene_expr(e), obj(o)
24 rel eval_objs(e, o) = unique_expr(e, f), eval_objs(f, o)
25 rel eval_objs(e, o) = filter_size_expr(e, f, s), eval_objs(f, o), size(o, s)
26 rel eval_objs(e, o) = filter_color_expr(e, f, c), eval_objs(f, o), color(o, c)
27 rel eval_objs(e, o) = filter_material_expr(e, f, m), eval_objs(f, o), material(o, m)
28 rel eval_objs(e, o) = filter_shape_expr(e, f, s), eval_objs(f, o), shape(o, s)
29 rel eval_objs(e, o) = and_expr(e, f1, f2), eval_objs(f1, o), eval_objs(f2, o)
30 rel eval_objs(e, o) = or_expr(e, f1, f2) and (eval_objs(f1, o) or eval_objs(f2, o))
31
32 // same attribute
33 rel eval_objs(e, o) = same_size_expr(e, f),
34                      eval_objs(f, p), size(p, c), size(o, c), o != p
35 rel eval_objs(e, o) = same_color_expr(e, f),
36                      eval_objs(f, p), color(p, c), color(o, c), o != p
37 rel eval_objs(e, o) = same_material_expr(e, f),
38                      eval_objs(f, p), material(p, m), material(o, m), o != p
39 rel eval_objs(e, o) = same_shape_expr(e, f),
40                      eval_objs(f, p), shape(p, s), shape(o, s), o != p
41
42 // relation
43 rel relate("right", p, o) = relate("left", o, p)
44 rel relate("front", p, o) = relate("behind", o, p)
45 rel eval_objs(e, o) = relate_expr(e, f, r),
46                      eval_objs(f, p), relate(r, p, o), o != p
47
48 // eval query
49 rel eval_query(e, s) = query_size_expr(e, f), eval_objs(f, o), size(o, s)
50 rel eval_query(e, c) = query_color_expr(e, f), eval_objs(f, o), color(o, c)
51 rel eval_query(e, m) = query_material_expr(e, f), eval_objs(f, o), material(o, m)
52 rel eval_query(e, s) = query_shape_expr(e, f), eval_objs(f, o), shape(o, s)

```